Lab 3: Architecture Justification

Incident Escalation & On-Call Rotation Engine (Problem Statement #48)

Name: Vrunda C | SRN: PES1UG24CS541 | Section: I

Architecture Selection

We chose **Microservices Architecture** for the Incident Escalation & On-Call Rotation Engine System.

Architectural Choice

The system is split into small, independently deployable services, each owning one responsibility: Alert Ingestion Service, Rotation Schedule Service, Escalation Engine, Notification Dispatcher and Incident & Post-Mortem Service, with an On-Call Web & Mobile Console as the client and an Incident & Schedule Database for persistence. Services communicate through well-defined interfaces (HTTPS REST, gRPC and asynchronous message queues). Layered architecture was rejected because every alert would pass through all layers and the whole application would scale as one unit. Client-Server was rejected because a single central server is a single point of failure, which is unacceptable for a paging system.

Reason 1: Independent scaling for alert storms

During a major outage, monitoring tools can send hundreds of alerts per minute while schedule edits and post-mortems stay rare. With microservices, only the Alert Ingestion Service, Escalation Engine and Notification Dispatcher are scaled out with extra instances. Rarely used services keep their normal size, so resources go exactly where the load is.

Reason 2: Fault isolation for high availability

The critical path (ingest → escalate → notify) must keep working even if another part fails (NFR-002: 99.95% availability, failover within 60 seconds). If the Incident & Post-Mortem Service or the Console crashes, alerts are still routed and the 5-minute escalation to the secondary engineer (FR-001) still fires. Escalation timers are persisted in the database, so a restarted Escalation Engine instance resumes pending escalations without losing any incident.

Security Advantage

Each service exposes only a narrow interface and runs with least privilege. The Alert Ingestion Service accepts webhooks only over HTTPS with HMAC-signed payloads and rejects unauthenticated requests. Console users are authenticated with tokens over HTTPS. Internal service-to-service traffic uses mutual TLS, and each service connects to the database with its own credentials and schema. As a result, a compromise of one service (for example the Console) cannot directly page engineers or change escalation timers, which limits the blast radius of an attack.

Performance Benefit

The Alert Ingestion Service, Escalation Engine and Notification Dispatcher are decoupled by asynchronous message queues. Ingestion acknowledges an alert immediately, and the Dispatcher sends phone, SMS, email and webhook notifications in parallel through the Notification Gateway. Because no step waits on slower services such as post-mortem reporting, and the Dispatcher can be scaled horizontally, alert dispatch initiates within 3 seconds of ingestion even under peak load (NFR-001).